

Extract



Interaktive Medien 3

nick.schneeberger@fhgr.ch

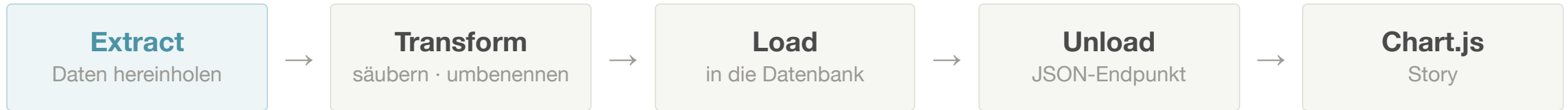
Inhalt

- Was heisst Extract?
- Datei oder Live-API?
- JSON, API, CSV und Sensorbox lesen
- Die eigene Quelle prüfen

Danach folgen vier Code-Alongs – eines pro Quelle.

Am Ende des Tages solltet ihr eure eigene Datenquelle ausgewählt haben.

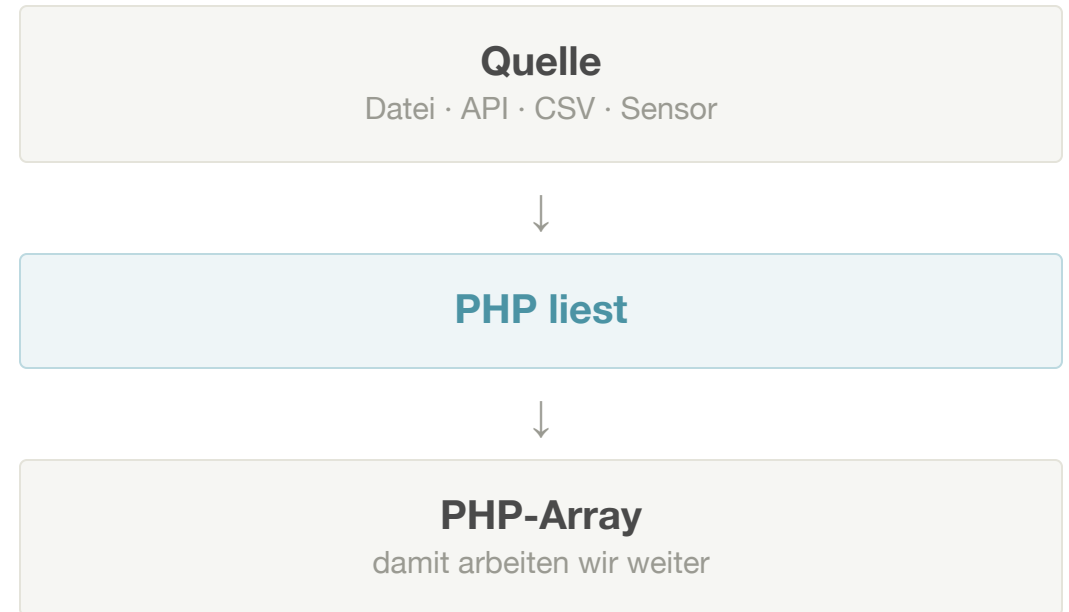
Wo wir stehen



Was heisst Extract?

Extract heisst lesen – die Daten kommen von aussen in dein PHP-Programm.

- Ziel: Quelle → **PHP-Array**
- Nicht dazu gehört: umbenennen und filtern
- Nicht dazu gehört: speichern
- Nicht dazu gehört: einen eigenen Endpunkt bauen



Am Ende jedes Code-Alongs steht dieselbe Kontrollfrage: Habe ich ein PHP-Array?

Vier Quellen, ein Ergebnis

JSON-Datei

Einmal heruntergeladen und liegt bei euch im Ordner.

Live-API

Wird bei jedem Aufruf frisch aus dem Internet geholt.

CSV-Datei

Eine Tabelle als Text, meist aus einem offenen Datenportal.

Sensorbox

Eine kleine eigene API, die im Kursraum steht und misst.

Ein Google Sheet lässt sich als CSV veröffentlichen und wird damit zur letzten Variante.

Woher kommen die Daten?

Herunterladen oder live abrufen

Live abrufen und sammeln

Das Programm holt die Daten bei jedem Aufruf neu.

- Immer aktuell
- Nötig, wenn ihr über die Zeit sammelt
- Ohne Internet keine Daten
- Die Quelle kann sich ändern oder ausfallen
- Am Anfang habt ihr sehr wenige Daten

Einmal herunterladen

Der Datensatz liegt als Datei bei euch im Projekt.

- Funktioniert immer und überall
- Grosse Datenmengen
- Ihr könnt die Daten jederzeit ansehen
- Datensatz veraltet mit der Zeit

Unser Beispiel: Hitzesommer

- Drei Städte: **Bern, Zürich, Chur**
- Tägliche Höchsttemperatur seit **1940**
- Über **31'000** Tage pro Stadt
- Quelle ist die Archiv-API von Open-Meteo
- Die Antwort habt ihr als Datei gespeichert

DATA/BERN.JSON

```
{
  "latitude": 46.95,
  "daily": {
    "time": ["1940-01-01", "1940-01-02"],
    "temperature_2m_max": [-0.7, 2.1]
  }
}
```

Dieselbe API liefert auf einem anderen Endpunkt auch die aktuellen Werte von heute.

Quelle: Open-Meteo (open-meteo.com), freie Nutzung ohne API-Key.

Von der Quelle ins PHP-Array

Vier Techniken, dasselbe Ziel

Eine JSON-Datei lesen

```
<?php
header('Content-Type: text/plain; charset=utf-8');

$json = file_get_contents('data/bern.json');    // Datei als Text
$data = json_decode($json, true);              // Text als PHP-Array

echo count($data['daily']['time']);
```

- `file_get_contents` liest die ganze Datei als **String**
- `json_decode` macht daraus ein **PHP-Array**
- Der Pfad gilt relativ zur PHP-Datei, die gerade läuft

Das `true` in `json_decode` ist kein Detail.

Ohne dieses Argument bekommst du Objekte (`$data->daily`) statt Arrays (`$data['daily']`).

Zwei Listen, die zusammengehören

Open-Meteo liefert keine Liste von Datensätzen, sondern **zwei gleich lange Listen**.

SO KOMMEN DIE DATEN AN

```
"time": ["1940-01-01", "1940-01-02"]  
"temperature_2m_max": [-0.7, 2.1]
```

SO LESEN WIR SIE

```
$dates = $data['daily']['time'];  
$temps = $data['daily']['temperature_2m_max'];  
  
for ($i = 0; $i < 5; $i++) {  
    echo $dates[$i] . ': ' . $temps[$i] . " °C\n";  
}
```

- `time[0]` gehört zu `temperature_2m_max[0]`
- Weil beide Listen denselben Index brauchen, passt hier `for` besser als `foreach`
- In unsere eigene Struktur bringen wir das erst im nächsten Schritt

Eine Live-API lesen

```
// Vorbereiteter Helfer: holt eine URL und gibt die Antwort als PHP-Array zurück.
function fetchJson(string $url): array {
    $ch = curl_init($url);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    curl_setopt($ch, CURLOPT_TIMEOUT, 10);
    $response = curl_exec($ch);
    return json_decode($response, true);
}

$url = 'https://api.open-meteo.com/v1/forecast?latitude=46.948&longitude=7.447'
    . '&hourly=temperature_2m&forecast_days=1&timezone=Europe/Zurich';
$data = fetchJson($url);
```

- Den cURL-Code musst du nicht auswendig können – nur benutzen
- Das Ergebnis ist wieder ein PHP-Array mit zwei parallelen Listen

cURL ist ein **ausgehender** Aufruf von unserem Server an eine fremde API.

Eine CSV-Datei lesen

```
$handle = fopen('data/attacks.csv', 'r');

// Kopfzeile = Spaltennamen
$header = array_map('trim', fgetcsv($handle, null, ',', '"', ''));

$attacks = [];
while (($row = fgetcsv($handle, null, ',', '"', '')) !== false) {
    if ($row[0] === '') {
        continue;           // leere Zeile überspringen
    }
    $attacks[] = array_combine($header, $row);
}
fclose($handle);
```

- CSV ist eine **Tabelle als Text**
- Die erste Zeile enthält die Spaltennamen
- `fgetcsv` liest genau **eine** Zeile
- `array_combine` verbindet Spaltennamen und Werte
- So entsteht ein Datensatz pro Zeile

Die Argumente `',' '"'` sind die Standardwerte und stehen nur da, damit neuere PHP-Versionen nicht warnen.

Echte Daten sind unordentlich

Was im Shark-Datensatz steckt

- Rund **25'000** Zeilen
- Davon etwa **8'700** echte Vorfälle
- Der Rest sind leere Zeilen
- Spaltennamen mit Leerzeichen wie `"Species "`
- Teils doppelte Spalten

Was wir dagegen tun

- `trim()` räumt die Spaltennamen auf
- `continue` überspringt leere Zeilen
- Sonst nichts

Hier räumen wir nur so weit auf, dass das **Einlesen** funktioniert.

Datumsformate, fehlende Werte und Kategorien kommen im nächsten Schritt dran: Transform.

Die Sensorbox im Raum

- Die Box misst laufend, zum Beispiel Temperatur, Luftfeuchtigkeit und CO₂
- Sie stellt ihre Werte über HTTP als JSON bereit
- Technisch ist sie damit einfach eine kleine **eigene API**
- Wir holen die Daten mit demselben `fetchJson()` - Helfer
- Wer selber misst, besitzt die Quelle und kann die Datenfrage frei stellen
- Wer selber misst, muss früh anfangen zu sammeln
- Hardware und Programmierung der Box sind nicht Inhalt dieses Kurses

Die anderen drei Quellen liefern **fremde** Daten.

Hier misst ihr **selber** – der Unterschied ist inhaltlich gross und technisch klein.

Der Extract ändert sich, das Ergebnis nicht

Quelle	Technik in PHP	Ergebnis
JSON-Datei	<code>file_get_contents</code> + <code>json_decode</code>	PHP-Array
Live-API	<code>fetchJson</code> (cURL)	PHP-Array
CSV-Datei	<code>fopen</code> + <code>fgetcsv</code>	PHP-Array
Sensor-API	<code>fetchJson</code> (cURL)	PHP-Array

Ab dem PHP-Array ist der weitere Weg für alle Projekte derselbe.

Die eigene Quelle prüfen

Bevor ihr euch festlegt

Woran ihr eine brauchbare Quelle erkennt

- **Zugang:** ohne Anmeldung und ohne API-Key erreichbar
- **Format:** JSON oder CSV, nicht PDF und nicht Bild
- **Menge:** genug Datensätze für eine Aussage
- **Zeitraum:** passt zu eurer Datenfrage
- **Felder:** dokumentiert oder selbsterklärend benannt
- **Herkunft:** Wer hat gemessen, wann und wie?
- **Lizenz:** Weiterverwendung ist erlaubt
- **Grenzen:** Was fehlt in diesen Daten?

Prüft die Quelle, indem ihr sie einmal im Browser öffnet und wirklich hineinschaut.

Eine schöne Projektseite sagt nichts darüber aus, ob die Daten brauchbar sind.

Häufige Fehler

- **Datei nicht gefunden:** der Pfad gilt relativ zur laufenden PHP-Datei
- **Objekte statt Arrays:** das `true` bei `json_decode` vergessen
- **Alles auf einer Zeile:** der `Content-Type`-Header fehlt
- **Leere Antwort von der API:** zuerst die URL im Browser aufrufen
- **Undefined key:** Schlüssel falsch geschrieben oder eine Ebene übersprungen
- **Verschobene Werte:** die Kopfzeile der CSV mitgezählt
- **Spalte existiert nicht:** Leerzeichen im Spaltennamen
- **Nichts passiert:** die Datei per Doppelklick statt über `http://` geöffnet

Erster Griff bei jedem Problem: `var_dump` oder `print_r` auf die Variable, statt zu raten.

Was jetzt kommt

Code-Alongs

- `06_json_lesen` – statische Datei
- `07_api_lesen` – Live-API
- `08_csv_lesen` – CSV-Datei
- `09_sensor_lesen` – Sensorbox

Danach im Projekt

- Datenquellen recherchieren und prüfen
- Eigene Quelle als PHP-Array einlesen
- Fallback festlegen

Meilenstein M3: Am Ende des Tages steht euer Datensatz.

Zum Mitnehmen

Egal woher die Daten kommen –
am Ende steht ein PHP-Array.

Extract holt die Daten nur herein: lesen, prüfen, weitergeben.

Nächster Schritt: Code-Along 06 – JSON lesen